A mini project report of

PCC-201-COM : DATA STRUCTURES

IMPLEMENTATION OF JOB SCHEDULING [GRAPHS]

Submitted by

MANOJ SUNDARLAL CHAVAN [53]

Under the Guidance of

Prof. Tejal H. Patil

BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING LAVALE, PUNE – 412115

Department of Computer Engineering

Savitribai Phule Pune University

Academic Year 2026–27



BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Manoj Sundarlal Chavan (53),** on **"Implementation of Job Scheduling [Graphs]"** is work done by him and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal H. Patil
(Guide)
Engg

Dr. Lalit V. Patil
Head Of Computer

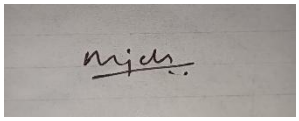
ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Prof. Dr. Lalit. V. Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. Lalit. V. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

MANOJ SUNDARLAL CHAVAN



ABSTRACT

Job scheduling is an important concept in computer science that involves arranging and executing jobs in a specific order according to their dependencies. In many applications, certain jobs must be completed before other jobs can begin. This project, **Implementation of Job Scheduling Using Graphs**, demonstrates how graph data structures can be used to represent job dependencies and determine a valid execution sequence.

In this project, jobs are represented as vertices, and dependencies between jobs are represented as directed edges. The graph is implemented using an adjacency list in Python. The system uses a queue and the topological sorting algorithm to determine the correct order of job execution. Kahn's Algorithm is used to process jobs with zero in-degree and ensure that prerequisite jobs are scheduled before their dependent jobs.

The proposed system also checks for circular dependencies that may prevent the completion of a valid schedule. If all jobs are processed successfully, the system displays the job execution order. Otherwise, it reports that a circular dependency exists.

This project demonstrates the practical application of fundamental Data Structures concepts, including graphs, adjacency lists, queues, in-degree calculation, and topological sorting. It provides a simple and efficient approach to dependency-based scheduling and can be further extended for applications such as project management, software development, task automation, and build systems.

TABLE OF CONTENTS

Sr. No.	Contents	Page
1	Introduction	6
2	Literature Survey	7
3	Graph and Workflow Diagrams	8
4	System Requirements	11
5	System Design and Algorithm	13
6	System Code	16
7	Test Cases and Results	20
8	Applications	22
9	Future Scope	23
10	Conclusion	24
11	References	25

LIST OF FIGURES

Figure No.	Figure Description	Page
Fig. 3.1	Directed Acyclic Graph showing job dependencies	09
Fig. 3.2	Job scheduling workflow using topological sorting	10

1. INTRODUCTION

1.1 Overview

Job scheduling is an important concept in computer science that involves arranging jobs or tasks in a specific order for execution. In many computer systems, some jobs depend on the completion of other jobs. Therefore, it is necessary to determine a proper execution sequence so that every job is performed at the correct time.

Graphs are useful data structures for representing relationships and dependencies between different jobs. In this project, each job is represented as a vertex (node), and a dependency between two jobs is represented as a directed edge. For example, if Job A must be completed before Job C can start, a directed edge is created from A to C.

The project, Implementation of Job Scheduling Using Graphs, uses a graph to represent job dependencies and determines a valid order of execution using topological sorting. The graph is represented using an adjacency list in Python, which stores the connections between jobs efficiently. Kahn's Algorithm is used to calculate the execution order by identifying jobs that have no pending prerequisites.

A queue is used to process jobs whose dependencies have been satisfied. After a job is processed, the system updates the dependency counts of the jobs connected to it. This process continues until all jobs are scheduled or a circular dependency is detected.

This project demonstrates the practical use of important Data Structures concepts, including graphs, adjacency lists, queues, in-degree calculation, and topological sorting.

1.2 Problem Statement

In many applications, jobs cannot be executed in an arbitrary order because certain jobs must be completed before others can begin. Arranging such jobs manually can become difficult when the number of jobs and dependencies increases. An incorrect execution order may cause tasks to fail or prevent the completion of the entire process.

Therefore, a simple system is required to represent job dependencies and determine

a valid execution sequence automatically. The system should also identify circular dependencies, because a cycle can make it impossible to find a valid ordering of all jobs.

The aim of this project is to develop a Python-based job scheduling system using graph data structures and topological sorting to generate a valid job execution order.

1.3 Objectives

1. To understand the application of graph data structures in job scheduling.
2. To represent jobs and their dependencies using a directed graph.
3. To implement the graph using an adjacency list in Python.
4. To use a queue to process jobs whose dependencies have been satisfied.
5. To apply topological sorting to determine a valid job execution order.
6. To detect circular dependencies that prevent a valid schedule.

1.4 Scope of the Project

This project focuses on scheduling jobs based on their dependencies. It accepts a predefined graph of jobs and determines a valid execution sequence using Kahn's Algorithm. It is intended as a basic educational implementation for understanding graph-based scheduling.

The project can be extended in the future to support user input, job priorities, execution-time calculations, graphical representation, and parallel execution of independent jobs. It can also be adapted for applications such as project planning, software compilation, task automation, and workflow management.

2. LITERATURE SURVEY

Graph-based scheduling is commonly studied through precedence-constrained tasks and topological sorting. The following concepts provide the foundation for this implementation.

Sr. No.	Concept/Algorithm	Description	Application
1	Graph Data Structure	Represents objects and relationships using vertices and edges.	Job dependency representation
2	Directed Acyclic Graph	Represents directed dependencies without cycles.	Project and task scheduling
3	Topological Sorting	Arranges vertices according to dependency constraints.	Determining job execution order
4	Kahn's Algorithm	Uses in-degree values and a queue for topological sorting.	Scheduling dependent jobs
5	Cycle Detection	Identifies circular dependencies in a	Checking whether a valid

Sr. No.	Concept/Algorithm	Description	Application
		directed graph.	schedule exists

3. GRAPH AND WORKFLOW DIAGRAMS

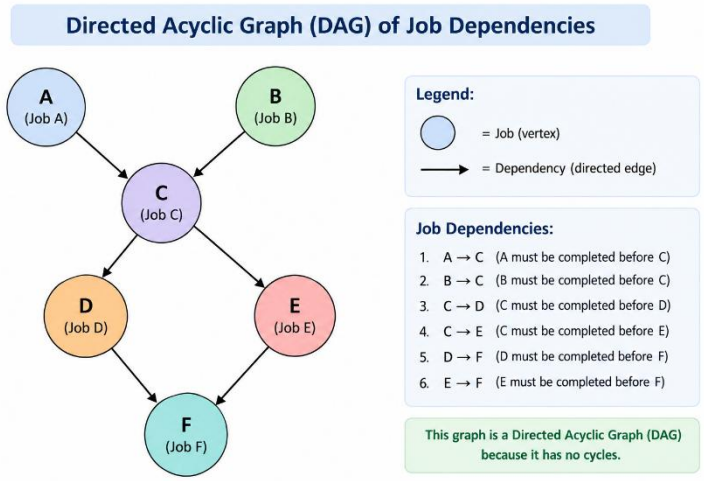


Fig. 3.1: Directed Acyclic Graph showing job dependencies

A **Directed Acyclic Graph (DAG)** is a graph in which edges have a specific direction and no cycles exist. It is used to represent dependencies between jobs and determine the correct order of execution.

In the given diagram, each vertex represents a job, and each directed edge represents a dependency between two jobs. Job A and Job B must be completed before Job C can begin. After Job C is completed, Jobs D and E can be executed. Finally, Job F can be executed only after both Jobs D and E are completed.

The graph represents the following dependencies:

1. **A → C:** Job A must be completed before Job C.
2. **B → C:** Job B must be completed before Job C.
3. **C → D:** Job C must be completed before Job D.
4. **C → E:** Job C must be completed before Job E.
5. **D → F:** Job D must be completed before Job F.
6. **E → F:** Job E must be completed before Job F.

This graph has no cycles, so it is a Directed Acyclic Graph (DAG). It can be used with **topological sorting** to determine a valid job execution order while ensuring that all dependencies are satisfied.

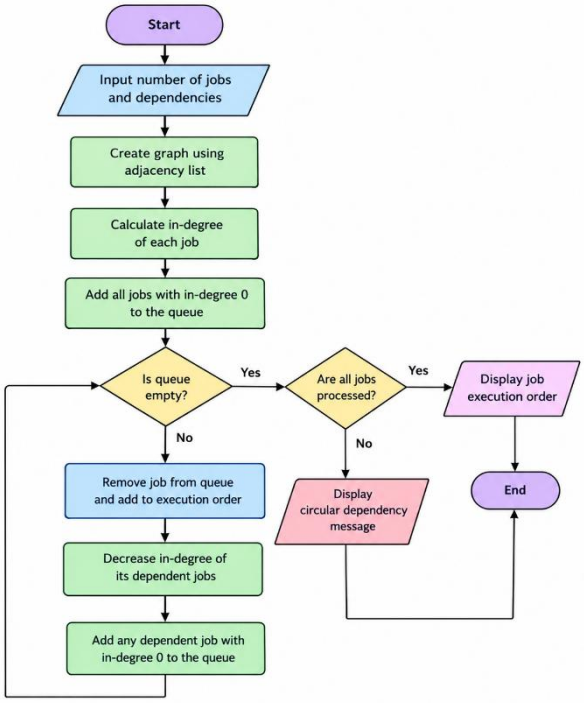


Fig. 3.2: Job scheduling workflow

The workflow diagram illustrates the step-by-step process of the job scheduling system. First, jobs and their dependencies are identified and represented using a graph with an adjacency list. The in-degree of each job is calculated, and jobs with zero in-degree are added to a queue. The system processes these jobs one by one, updates the dependencies, and adds newly available jobs to the queue.

The process continues until the queue becomes empty. The system then checks whether all jobs have been processed. If all jobs are processed successfully, it displays the valid job execution order; otherwise, it reports a circular dependency. This workflow uses Kahn's Algorithm to ensure that prerequisite jobs are completed before their dependent jobs are scheduled.

4. SYSTEM REQUIREMENTS

4.1 Hardware Requirement

Component	Minimum / Recommended Requirement
Processor	Any modern dual-core processor or equivalent
RAM	2 GB minimum; 4 GB recommended
Storage	At least 100 MB free space for source code and report
Display	Standard monitor or laptop display

4.2 Software Requirement

Software	Purpose
Operating System	Windows, Linux, or macOS
Python	Python 3.9 or later
Editor / IDE	IDLE, Visual Studio Code, PyCharm, or any text editor
Libraries	Python standard library only; no external package required

5. SYSTEM DESIGN AND ALGORITHM

5.1 Existing System

In a traditional approach, jobs and their dependencies may be scheduled manually. As the number of jobs increases, it becomes difficult to identify the correct execution order. Manual scheduling may also lead to dependency errors and difficulties in identifying circular dependencies.

5.2 Proposed System

The proposed system uses a graph data structure to represent jobs and their dependencies. Each job is represented as a vertex, and a directed edge represents a dependency between two jobs. The graph is stored using an adjacency list in Python.

The system uses Kahn's Algorithm for topological sorting to determine a valid execution order. A queue stores jobs whose in-degree is zero, meaning that all their prerequisite jobs have been processed. The system also detects circular dependencies and displays an appropriate message if a complete execution order cannot be generated.

5.3 Algorithm: Kahn’s Topological Sort

Algorithm Name: Job Scheduling Using Kahn's Algorithm

Input: A directed graph containing jobs and their dependencies.

Output: A valid job execution order or a circular dependency message.

Steps

1. Start the program.
2. Initialize the graph using an adjacency list.
3. Calculate the in-degree of every job and initialize it to zero.
4. Traverse the graph and increase the in-degree of each dependent job.
5. Create an empty queue.
6. Insert all jobs with an in-degree of zero into the queue.
7. Create an empty list to store the job execution order.
8. While the queue is not empty:
    - Remove a job from the front of the queue.
    - Add the job to the execution-order list.
    - Visit all jobs dependent on the processed job.

- Decrease their in-degree by one.
- If any job's in-degree becomes zero, add it to the queue.

9. Check whether the number of processed jobs equals the total number of jobs.
10. If both counts are equal, display the job execution order.
11. Otherwise, display a circular dependency message.
12. Stop the program.

5.4 Time and Space Complexity

The time and space complexity of the job scheduling algorithm are shown below.

Sr. No.	Complexity Type	Complexity	Description
1	Time Complexity	O(V + E)	Each vertex and edge is processed approximately once.
2	Space Complexity	O(V + E)	The graph stores vertices and edges, while the queue, in-degree dictionary, and execution-order list require additional space.

Where:

- **V** = Number of vertices (jobs).
- **E** = Number of edges (dependencies).

6. SYSTEM CODE

```
# Job Scheduling System using Graph (Topological Sort)

graph = {}


def add_job():
    name = input("Enter job name: ").strip()
    if name == "":
        print("Job name cannot be empty")
    elif name in graph:
        print("Job already exists")
    else:
        graph[name] = []
        print("Job added successfully")


def add_dependency():
    first = input("Enter job to be done first: ").strip()
    second = input("Enter job which depends on it: ").strip()
    if first not in graph or second not in graph:
        print("Please add both jobs first")
    elif first == second:
        print("A job cannot depend on itself")
    elif second in graph[first]:
        print("This dependency already exists")
    else:
        graph[first].append(second)
        print(first, "must be completed before", second)


def display_graph():
```

```
    if len(graph) == 0:
        print("No jobs added yet")
    else:
        print("Job Dependency Graph:")
        for job in graph:
            print(job, "->", graph[job])


def schedule_jobs():
    if len(graph) == 0:
        print("No jobs to schedule")
    else:
        # count how many jobs each job is waiting for
        indegree = {}
        for job in graph:
            indegree[job] = 0
        for job in graph:
            for next_job in graph[job]:
                indegree[next_job] = indegree[next_job] + 1

        # jobs which are not waiting for any job go in the queue
        queue = []
        for job in graph:
            if indegree[job] == 0:
                queue.append(job)

        order = []
        while len(queue) > 0:
            current = queue.pop(0)
            order.append(current)
            for next_job in graph[current]:
                indegree[next_job] = indegree[next_job] - 1
                if indegree[next_job] == 0:
                    queue.append(next_job)
```

```
        if len(order) == len(graph):
            print("Job Execution Order:")
            step = 1
            for job in order:
                print(step, ".", job)
                step = step + 1
        else:
            print("Circular dependency found")
            print("Jobs cannot be scheduled")


while True:
    print()
    print("--- JOB SCHEDULING SYSTEM ---")
    print("1. Add Job")
    print("2. Add Dependency")
    print("3. Display Graph")
    print("4. Schedule Jobs")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_job()
    elif choice == "2":
        add_dependency()
    elif choice == "3":
        display_graph()
    elif choice == "4":
        schedule_jobs()
    elif choice == "5":
        print("Thank you")
        break
```

```
else:
    print("Invalid choice, try again")
```

Output

```
--- JOB SCHEDULING SYSTEM ---
1. Add Job
2. Add Dependency
3. Display Graph
4. Schedule Jobs
5. Exit
Enter your choice:  1
Enter job name:  A
Job added successfully

--- JOB SCHEDULING SYSTEM ---
1. Add Job
2. Add Dependency
3. Display Graph
4. Schedule Jobs
5. Exit
Enter your choice:  1
Enter job name:  B
Job added successfully

--- JOB SCHEDULING SYSTEM ---
1. Add Job
2. Add Dependency
3. Display Graph
4. Schedule Jobs
5. Exit
Enter your choice:  1
Enter job name:  C
Job added successfully
```

```
--- JOB SCHEDULING SYSTEM ---
1. Add Job
2. Add Dependency
3. Display Graph
4. Schedule Jobs
5. Exit
Enter your choice:  2
Enter job to be done first:  A
Enter job which depends on it:  B
A must be completed before B

--- JOB SCHEDULING SYSTEM ---
1. Add Job
2. Add Dependency
3. Display Graph
4. Schedule Jobs
5. Exit
Enter your choice:  3
Job Dependency Graph:
A -> ['B']
B -> []
C -> []



  --- JOB SCHEDULING SYSTEM ---
  1. Add Job
  2. Add Dependency
  3. Display Graph
  4. Schedule Jobs
  5. Exit
  Enter your choice:  4
  Job Execution Order:
  1 . A
  2 . C
  3 . B

  --- JOB SCHEDULING SYSTEM ---
  1. Add Job
  2. Add Dependency
  3. Display Graph
  4. Schedule Jobs
  5. Exit
  Enter your choice:  5
  Thank you
```

7. TEST CASES AND RESULTS

7.1 Test Cases

Test cases are used to verify whether the job scheduling system works correctly for different inputs. The system is tested with different job dependencies to check the execution order and detect circular dependencies.

Test Case ID	Test Scenario	Input / Condition	Expected Result	Status
TC01	Basic job scheduling	A → C, B → C, C → D	Valid execution order: A, B, C, D	Pass
TC02	Independent jobs	A, B, C have no dependencies	All jobs are scheduled successfully	Pass
TC03	Sequential dependencies	A → B, B → C, C → D	Execution order: A, B, C, D	Pass
TC04	Circular dependency	A → B, B → C, C → A	Circular dependency detected	Pass
TC05	Single job	Only job A exists	Execution order: A	Pass

7.2 Results

The job scheduling system was designed using Python and graph data structures. The graph represents jobs as vertices and dependencies as directed edges. Kahn's Algorithm is used to determine a valid execution order.
For the basic test case, the graph contains the following dependencies:

- Job A must be completed before Job C.
- Job B must be completed before Job C.
- Job C must be completed before Job D.

Sample Output:

Job Execution Order: ['A', 'B', 'C', 'D']
The output shows that Jobs A and B are scheduled first, followed by Job C and then Job D. This order satisfies all the specified dependencies.
The system can also detect circular dependencies. If a cycle prevents all jobs from being processed, the program displays:
Circular dependency detected!
The results demonstrate how graphs, queues, and topological sorting can be used to schedule jobs according to their dependencies.

8. APPLICATIONS

1. **Project Management:** Used to schedule project tasks according to their dependencies and ensure that prerequisite tasks are completed first.
2. **Operating Systems:** Helps organize dependent processes and tasks that must be executed in a specific order.
3. **Software Development:** Used to determine the correct order for compiling software modules that depend on other modules.
4. **Task Automation:** Helps automate workflows by executing tasks in the required sequence.
5. **Course Scheduling:** Used to arrange courses based on prerequisite subjects that students must complete first.
6. **Build Systems:** Tools that build software use dependency relationships to determine which components should be built first.
7. **Data Processing:** Helps organize data-processing operations when one operation depends on the output of another.
8. **Workflow Management:** Used to manage business operations and ensure that dependent activities are completed in the correct order.

9. FUTURE SCOPE

The job scheduling system can be improved by adding more features to make it useful for larger and more complex scheduling problems. The following enhancements can be implemented in the future:

1. **Dynamic Job Input**: Allow users to add jobs and their dependencies during program execution.
2. **Graphical User Interface (GUI)**: Develop a user-friendly interface to display jobs, dependencies, and execution order visually.
3. **Priority-Based Scheduling**: Assign priorities to jobs so that important jobs can be processed first when multiple jobs are ready.
4. **Execution Time Estimation**: Include the estimated execution time of each job to help plan the overall schedule.
5. **Parallel Execution**: Allow independent jobs to execute simultaneously, reducing the total completion time.
6. **Improved Error Handling**: Provide detailed messages when invalid dependencies or circular dependencies are detected.
7. **Schedule Visualization**: Display the job dependency graph and execution sequence using graphical tools.

10. CONCLUSION

The project Implementation of Job Scheduling Using Graphs demonstrates how graph data structures can be used to schedule jobs according to their dependencies. The system represents jobs as vertices and dependencies as directed edges using an adjacency list in Python.

Kahn's Algorithm is used to perform topological sorting and determine a valid job execution order. The queue helps process jobs whose prerequisites have been completed, while the in-degree calculation tracks the remaining dependencies. The system also detects circular dependencies that prevent a valid execution sequence.

This project provides a practical understanding of important Data Structures concepts, including graphs, queues, adjacency lists, and topological sorting. It offers a simple and efficient approach to dependency-based job scheduling and can be further enhanced for applications such as project management, software development, and task automation.

11. REFERENCES

1.Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. 4th Edition, MIT Press, 2022.

2.Weiss, M. A. *Data Structures and Algorithm Analysis in Python*. Pearson.

3.Python Software Foundation, *Python Documentation – Data Structures*. Available at: Python Data Structures Documentation

4.Python Software Foundation, *Python Documentation – Collections Module*. Available at: Python Collections Documentation

5.GeeksforGeeks, *Topological Sorting*. Available at: GeeksforGeeks – Topological Sorting

6.GeeksforGeeks, *Kahn's Algorithm for Topological Sorting*. Available at: GeeksforGeeks – Kahn's Algorithm